BANKNIFTY Automatic Data Collector — Project Guide

Verified project notes • 15 October 2026

Project status: Automatic BANKNIFTY collection verified successfully **Last verified:** 15 October 2026, 11:45 IST **Project directory:** `~/trading_project/nse_stockoption_csv`

1. What the project does

The collector saves BANKNIFTY option-chain and spot/futures snapshots to SQLite. A user-level `systemd` timer runs it every 15 minutes on weekdays.

The verified run at **2026-10-15 11:45:03 IST** completed successfully:

• 42 option contracts: 21 calls and 21 puts

• Underlying/reference price: 55,807.90

• Spot LTP: 55,809.80

• Futures symbol: `NSE:BANKNIFTY26OCTFUT`

• Futures LTP: 55,955.00

• Futures premium: 145.20 (about 0.26%)

• Both `snapshots` and `cash_futures_snapshots` had the new timestamp.

2. Important paths

Item	Path
Project folder	`~/trading_project/nse_stockoption_csv`
Collector script	`collect_index_data.py`
Virtual environment Python	`~/trading_project/.venv/bin/python`
Credentials environment file	`~/trading_project/.env`
Access token file	`~/trading_project/access.txt`
SQLite database	`database/stock_option_chain.db`
User service	`~/.config/systemd/user/nse-index-collector.service`
User timer	`~/.config/systemd/user/nse-index-collector.timer`

Security: Never paste, print, or add API credentials, secrets, or access tokens to this guide or logs.

3. Manual collection commands

Go to the project folder:

```
cd ~/trading_project/nse_stockoption_csv
```

Run BANKNIFTY collection:

```
~/trading_project/.venv/bin/python collect_index_data.py --symbol BANKNIFTY
```

Run NIFTY 50 manually (this does not change the scheduled BANKNIFTY service):

```
~/trading_project/.venv/bin/python collect_index_data.py --symbol NIFTY50
```

Open the interactive menu:

```
~/trading_project/.venv/bin/python collect_index_data.py
```

4. Automatic schedule

The active service runs `collect_index_data.py --symbol BANKNIFTY`.

Configured weekday times, in local machine time: **09:15, then every 15 minutes, through 15:30**. The timer explicitly lists 09:15, 09:30, 09:45, 10:00, 10:15, 10:30, 10:45, 11:00, 11:15, 11:30, 11:45, 12:00, 12:15, 12:30, 12:45, 13:00, 13:15, 13:30, 13:45, 14:00, 14:15, 14:30, 14:45, 15:00, 15:15 and 15:30, Monday through Friday.

Verified configuration:

- `Linger=yes` was enabled.
- `flock -n` is used to prevent overlapping runs.
- Service timeout: 180 seconds.
- `Persistent=true` is configured.

Limitations: The timer follows weekdays, not the official NSE holiday calendar, so it can attempt collection on exchange holidays. `Persistent=true` does not replay every missed interval. If a collection is still running at a later trigger, the lock can prevent that later run from collecting.

5. Check timer, service and logs

Timer status:

```
systemctl --user status nse-index-collector.timer --no-pager
```

Upcoming timer events:

```
systemctl --user list-timers --all
```

Service status:

```
systemctl --user status nse-index-collector.service --no-pager
```

Latest 50 log lines:

```
journalctl --user -u nse-index-collector.service -n 50 --no-pager
```

Follow logs live (press `Ctrl+C` to stop):

```
journalctl --user -u nse-index-collector.service -f
```

6. Verify latest database timestamps

Run from the project directory:

```
cd ~/trading_project/nse_stockoption_csv

python - <<'PY'
import sqlite3

db = "database/stock_option_chain.db"
with sqlite3.connect(db) as conn:
    for table in ("snapshots", "cash_futures_snapshots"):
        print(f"\\n{table}:")
        row = conn.execute(
            f"SELECT captured_at FROM {table} "
            "WHERE symbol='BANKNIFTY' "
            "ORDER BY captured_at DESC LIMIT 1"
        ).fetchone()
        print(row[0] if row else "No BANKNIFTY record")
PY
```

The latest timestamps in both tables should normally match. The verified run showed:

```
snapshots:
2026-10-15T11:45:03+05:30

cash_futures_snapshots:
2026-10-15T11:45:03+05:30
```

If the virtual environment is not active, replace `python` in the verification command with `~/trading_project/.venv/bin/python`.

7. Maintenance commands

After editing a unit file, reload systemd configuration:

```
systemctl --user daemon-reload
```

Restart the timer:

```
systemctl --user restart nse-index-collector.timer
```

Enable the timer:

```
systemctl --user enable --now nse-index-collector.timer
```

Temporarily stop automatic collection:

```
systemctl --user stop nse-index-collector.timer
```

Start it again:

```
systemctl --user start nse-index-collector.timer
```

Validate unit syntax (diagnostic command):

```
systemd-analyze --user verify \
  ~/.config/systemd/user/nse-index-collector.service \
  ~/.config/systemd/user/nse-index-collector.timer
```

Only run `sudo loginctl enable-linger "$USER"` if linger is not already enabled and you want the user service manager to remain active without an interactive login. Linger was already confirmed enabled for this setup.

8. Troubleshooting

If a new snapshot does not appear after a scheduled time:

1. Check the timer and next trigger with `systemctl --user list-timers --all`.

2. Check service result with `systemctl --user status nse-index-collector.service --no-pager`.

3. Read logs with `journalctl --user -u nse-index-collector.service -n 100 --no-pager`.

4. Verify the database timestamp using Section 6.

5. If logs mention authentication or token expiry, refresh the Fyers access token through your existing secure workflow. Do not put the token in shell commands that may be saved in history.

6. If you edited a unit file, run `systemctl --user daemon-reload`, restart the timer, then check status again.

9. Data caveats

• The collector reported **42 contracts (21 calls and 21 puts)**. PCR, OI walls and max pain are limited to the strikes returned by the API; do not treat them as a complete market-wide strike range.

• The timer does not automatically consult the NSE holiday calendar.

• A successful collection means data was collected and saved; it does not guarantee every market field is available or that a trading signal is profitable.

• This guide documents the verified BANKNIFTY scheduler. Manual NIFTY 50 collection is supported, but the active scheduled service is BANKNIFTY.

10. Completion checklist

■ BANKNIFTY collector ran successfully.

■ Service log reported `Collection successful`.

■ Option snapshot saved.

■ Cash/futures snapshot saved.

■ Latest database timestamps verified.

■ User-level timer enabled.

■ User linger enabled.

Conclusion: The basic BANKNIFTY automated collection workflow is operational. Let it collect for several sessions, then check for missing intervals and review the historical data before extending the project.